Assignment Report

SFWRENG 2AA4 (2026W)

Assignment 2

Author(s)

Devan Sandhu	sandhd7@mcmaster.ca	sandhd7
Nadeem Mohamed	mohamn84@mcmaster.ca	mohamn84
Billy Wu	wu897@mcmaster.ca	wu897
Nikhil Ranjith	ranjin1@mcmaster.ca	ranjin1
Vivek Patel	patev124@mcmaster.ca	patev124

GitHub URL

https://github.com/sparksavior/2aa4-team1

March 10, 2026

1 Executive Summary

Our team extended the Catan game simulator developed and designed previously into an interactive and observable gameplay experience. The objective of our current iteration was to introduce human and computer interaction as well as allowing humans to take turns alongside the computer. To achieve this we developed a command line interface supported by regex driven command parser. The system allows users to evaluate the game state, issue structural commands and control flow of the simulation. A major focus was kept on evolvability and maintainability so the integration adheres to OOP and SOLID principles. Beyond user input, a critical enhancement was the introduction of real time tracking and complex game mechanics visually. We integrated the simulator with a visualizer that generates and updates JSON representation of the Catan board. To ensure the human player remained fully informed of the computer's decisions, we implemented a step forward execution mode to pause the simulation between computer turns until user progression is authorized. Moreover, the Robber mechanism was integrated into the simulation which needed careful management of state changes to handle tile placement, dynamic resource stealing, etc. The entire system is backed by approximately 35 unit tests which incorporate boundary tests, and partitions tests, ensuring the simulator remains stable and scalable for future developments.

2 Requirements Traceability

Req ID	Status	Implemented in	Design considerations
R2.1	Complete	CommandParser.java, HumanPlayer.java	We utilized regex to perform command validations. Decoupled parsing logic from player class to enforce SRP.
R2.2	Complete	GameStateExporter.java, visualize/	Decoupled visualization from core engine to avoid muddying game logic. Relying on external scripts ensures visualizer runs asynchronously.
R2.3	Complete	GameStateExporter.java, visualize/state.json	JSON export triggers when board state is changed. Iterates over Paths and Intersections to accurately represent occupant ownership and types dynamically.
R2.4	Complete	Simulator.java	There is a stepMode flag toggled via Simulator constructor. Implemented blocking Scanner.nextLine() call that halts execution loop after a ComputerPlayer turn

			until the word “go” is inputted.
R2.5	Complete	Simulator.java, Robber.java	We have also made Robber movement logic independent (own class). Simulator will lead the multistep process during a roll of 7: such as evaluating player hand sizes, selecting a valid non desert Tile, etc.
R2.6	Complete	Demonstrator.java	Designed as clean entry point leveraging GameConfig. Includes multiline class comments explicitly detailing how application demonstrates required Assignment 2 functionalities.

3 Task 1: Getting Ready for Change

We developed and maintained 35 JUnit tests for the Catan Game system. It is organized into multiple classes such as BoardTest, PlayerTest, CommandParserTest, RobberTest, and SimulatorTest. It ensures the structural requirements of the board, game logic, player behaviour, dice rolling mechanism, CommandParser, and A2 features.

The tests include:

- Boundary testing: E.g. Validating minimum and maximum limits in Game Config
- Partition testing: E.g. Testing different cases of Player.canAfford, including resources (exact/insufficient), and missing resource types
- Behavioral testing: E.g. Verifying that settlement building succeeds when valid and fails when invalid. Ensures resources are consumed appropriately
- Structural testing: E.g. Verifying paths connect valid intersections and the board is set up correctly
- Range testing: E.g. Ensuring the dice yields a value between 2 and 12.
- Assignment 2 features including robber behaviour, and command execution outputs for core commands and build commands.

All tests pass successfully when executed using MVN tests, confirming the baseline behaviour of assignment 1. It further proves that the increased features did not break existing code.

Test coverage was measured using JaCoCo. The overall results were 72% line coverage and 35% branch coverage. This verifies the main game logic and any future additions can be easily validated with automated tests.

Catan Simulator | Sessions

Catan Simulator

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.assignment1.core		29%		24%	37	51	98	144	12	18	0	2
com.assignment1.player		51%		35%	43	69	84	161	13	28	1	3
com.assignment1.board		90%		42%	50	97	54	203	13	45	0	5
com.assignment1.command		54%		21%	15	28	22	54	0	11	0	5
com.assignment1		0%		n/a	2	2	5	5	2	2	1	1
com.assignment1.pieces		63%		n/a	3	7	5	14	3	7	2	5
com.assignment1.config		98%		90%	1	9	1	26	0	4	0	1
com.assignment1.enums		100%		n/a	0	5	0	27	0	5	0	5
Total	1,163 of 4,287	72%	186 of 289	35%	151	268	269	634	43	120	4	27

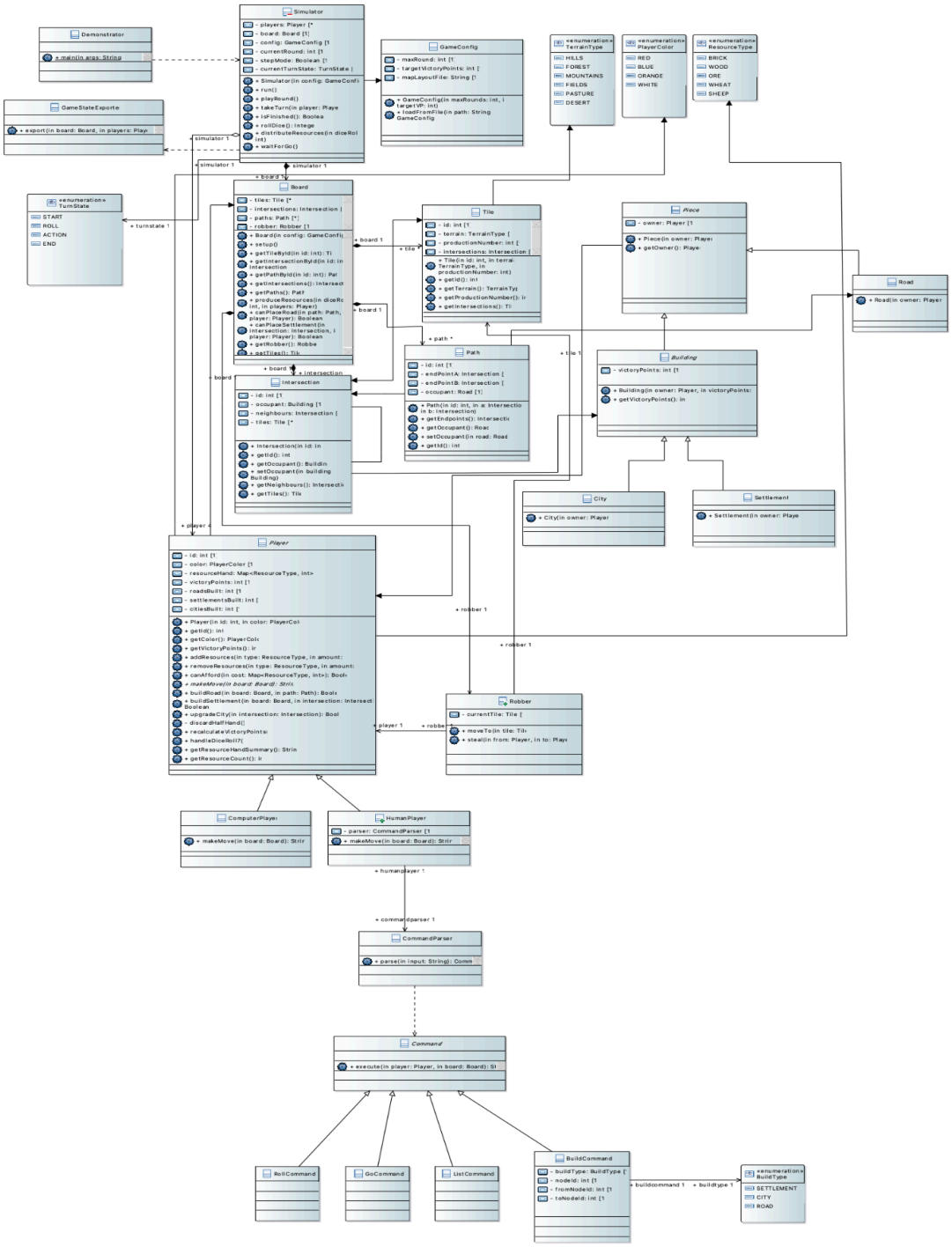
Created with JaCoCo 0.8.12.202403310830

The report shows that there was moderate coverage in core, player and command, while the config and board packages achieved high coverage (>90%). This coverage percentage ensures the verification of critical rules and advanced decision logic. The tests were executed without any failures, indicating that the Catan system is in a stable state. Any new features can be immediately validated ensuring the evolvability of the system.

---

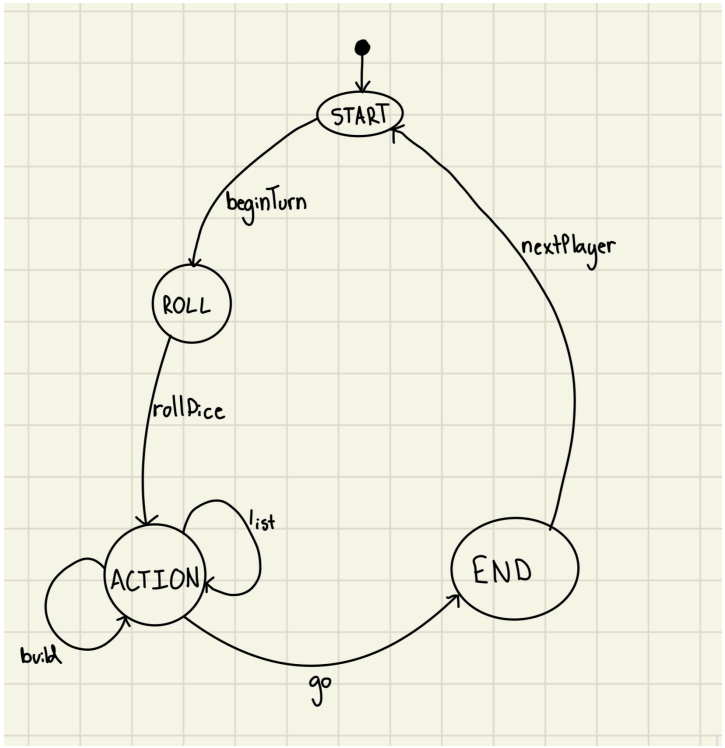
4 Design and Domain Modeling

***Please note** the below UML image was taken directly from the papyrus screenshot tool and some text cutoff has occurred. This was the only best option to ensure a clear image in the report. If you would like a clearer image you can view it in our repository here or view our Papyrus project zip here.



The UML Diagram for our project.

One of the main structural changes in our design was introducing inheritance into the player class. Before, the player class was responsible for all player operations regardless of if the player was human or computer, all players had the same behaviours. To ensure the program allowed for different types of players, we made Player an abstract class with subclasses HumanPlayer and ComputerPlayer both with the abstract method makeMove() which have their respective implementation using their own logic to determine how moves are made. This follows the DIP SOLID principle since we rely on abstracts such as the player class instead of concrete. This also supports the open closed principle since the player class is closed for modification but can also be extended if more player types are needed. Using the UML to make changes made the process of making changes simple and understandable. The structure of the UML makes it clear which attributes are going to remain shared and which will be implemented inside the subclasses. Having a plan with the UML made it easier since we didn't have to immediately modify the implementation and potentially ruin it, instead we were able to come up with a plan to tackle the changes one step at a time. Overall, modifying the design first helped clarify the architecture before implementing the change in code, reducing the risk of introducing inconsistencies and creating errors.



The Design Automata for a turn.

Automaton based modelling was useful because it provided a clear and structured way to represent the sequence of actions that a player can perform during a turn. The Catan rulebook describes the gameplay rules in words, explaining what actions players can take, but it does not put the exact order in which actions must occur in an easily understandable way which can be understood at a glance. This could make translating the rulebook directly implementation difficult. By modelling the player's turn using a simple automata, the valid states and moves of the player become understandable. Each state represents a stage of a player's turn, and transitions represent actions such as rolling the dice, building, or ending

the turn. This made it easier to ensure that only valid actions occur during the game. The automaton can also easily translate into code. The states defined in the automaton were implemented as states using enumeration and the transitions were implemented using conditional logic that checked whether a given action was valid for the current state. The automata helped organize the game logic and ensured that the program followed the rules of the game in an easy to understand and structured way.

5 Task 3: Reflection

Demonstrator Class Overview

The *Demonstrator* class serves as the entrypoint that demonstrates all key Assignment 2 features through a single run. When executed, it loads the game configuration and initializes a *Simulator* instance with Player 1 as a *HumanPlayer* and Player 2 to 4 as *ComputerPlayer* instances. During gameplay, the human player can interact with the game by entering commands such as *Roll*, *Go*, *List*, and *Build*, which are parsed using a dedicated *CommandParser* I/O class with regular expressions and executed through the Command design pattern. The demonstrator showcases the robber mechanism when a dice roll of 7 occurs, triggering card discarding, random robber placement, and resource stealing from qualifying players. Additionally, the *GameStateExporter* automatically writes the current board state to *visualize/state.json* after each action, enabling real-time visualization through integrated Python visualizer script. The demonstrator can optionally run in step-forward mode, pausing after each computer player's turn to allow the human player to observe AI actions before proceeding.

Integration and Error Handling

Challenges: What broke and how it was resolved

During the integration of Assignment 2 features, several components broke and required resolution. The first significant issue occurred with the *GameStateExporter* class when a teammate reported that the JSON file was not being generated when the game was run from different working directories. The problem stemmed from using relative file paths without proper path resolution. The resolution involved using Java's Paths API for robust path resolution and ensuring parent directories are created automatically, making the exporter work correctly regardless of the directory from which the application is launched.

Another fundamental breaking change occurred when refactoring the *Player* class from concrete to abstract to support human player functionality. This change broke all existing A1 code that instantiated *Player* directly, requiring a systematic migration to *ComputerPlayer* throughout the codebase. During the integration of step-forward functionality, an initial implementation violated the UML design that only *HumanPlayer* should use *CommandParser*. The resolution was to remove *CommandParser* from *Simulator* and implement step-forward using simple string matching, while ensuring that *HumanPlayer* properly composes and uses *CommandParser* for its own command parsing needs. This separation clarified the distinction between player actions and simulation controls.

The Benefit

The primary benefit of having unit tests during this phase was the confidence they provided when refactoring multiple components simultaneously. When error message implementation was added, the existing test suite served as a safety net that verified the core parsing and execution logic remained intact after the changes. This was especially valuable when the *Player* class was refactored from concrete to abstract, as the tests helped identify which parts of the codebase needed updates.

OO Mechanism and SOLID

The *GameStateExporter* class was designed following the Single Responsibility Principle, with the sole responsibility of exporting the current game state to a JSON file. This separation of concerns ensures that the exporter can be modified or replaced without affecting the core game logic. The class follows the Dependency Inversion Principle in that it depends on the Board abstraction rather than concrete implementation details.

In the meantime, the BuildCommand class demonstrates the use of polymorphism through Command pattern, where different build operations are encapsulated as a single *Command* interface. This allows *HumanPlayer* to treat all build commands uniformly. The class also follows the Open Closed Principle, as new build types or validation rules can be added without requiring changes to the class that use it.

The refactoring of *Player* from concrete to abstract demonstrates the Liskov Substitution Principle, as both *HumanPlayer* and *ComputerPlayer* can be used interchangeably wherever a *Player* is expected.

CommandParser and CommandParserTest

What broke and how it was resolved

The CommandParserTest broke when we refactored CommandParser to return specific command subclasses, such as the BuildCommand or RollCommand class. The subclasses didn't have getter methods for internal fields like nodeId or buildType which made assertions impossible. We resolved this by utilizing Java Reflection in CommandParserTest to inspect private fields of returned Command objects which allowed tests to pass without modifying another team member's code.

The Benefits

The primary benefit of having unit tests was to confidently swap the entire return type logic of the parser if needed. The tests were verifying that our regular expressions such as handling commas versus spaces in the Build road command were still functioning well after refactoring. So, the CommandParserTest served as a safety net during our development, ensuring structural class changes didn't affect the functionality.

OO Mechanisms and SOLID

The CommandParser.java had the sole responsibility of evaluating regular expressions and map string inputs to objects. We utilized polymorphism by returning the abstract Command base class rather than concrete implementations, which aligns closely with open close principle. If new commands are added to game later, the parser easily returns new Command subclasses without need to change core class. These principles align perfectly with our software design as keeping parsing isolated was the intended goal.

Helpful Tools and Integrated Toolchains

We utilized Maven for builds and JaCoCo for coverages. We could have used GitHub Actions which could have automated workflows for 35 unit test cases every time a team member pushed code. This avoids manual tests and faster development. Testing manually took a bit of time and automations would greatly help reduce the burden.

6 Roles and Responsibilities

The team members contributed equally to the deliverable.